Lecture 19 - Cloud Data Warehouses

- Present paper: VLDB 2020 Test of Time award for VLDB 2010 Paper on Dremel, the basis of BigQuery
- Interestingly, SIGMOD 2026 Test of Time award for SIGMOD 2016 paper on Snowflake
- Goal: to learn about cloud data warehouses like BigQuery
  - Putting together ideas from columnar databases/compression, parallel data processing
  - Plus a few other ideas...

Setup

- Interesting titbit: Mike F introduced the paper by saying that

> *""“eye-opening” to learn that Google engineers routinely analyzed massive data sets with processing throughputs in the range of 100 billion records per second"*

- But design to enable that wasn't trivial
- Major contributions:
  - SQL (instead of M/R)
  - Disaggregated compute/storage
  - Operate on data lakes ("in situ" analysis)
  - Serverless computing
  - Columnar encoding (but a novel variant for semi-structured data)

SQL

- As of 2010, MapReduce was still popular; Spark paper had yet to be published (OSDI 2012).
  - Spark SQL would not be introduced until 2014
- "*conventional wisdom at Google was “SQL doesn’t scale”, and with a few exceptions, had moved away from SQL almost completely*."
- Dremel reintroduced SQL - and showed that one can use SQL to analyze web-scale datasets.
  - Enable declarative *and interactive* analysis on large datasets
  - Original Dremel paper:

```sql
SELECT DocId AS Id,
  COUNT(Name.Language.Code) WITHIN Name AS Cnt,
  Name.Url + ',' + Name.Language.Code AS Str
FROM t
WHERE REGEXP(Name.Url, '^http') AND DocId < 20;
```

```
Id: 10                                  t1
Name
  Cnt: 2
  Language
    Str: 'http://A,en-us'
    Str: 'http://A,en'
Name
  Cnt: 0
```

```
message QueryResult {
  required int64 Id;
  repeated group Name {
    optional uint64 Cnt;
    repeated group Language {
      optional string Str; }}}
```

Figure 6: Sample query, its result, and output schema

-
  - "Think of a nested record as a labeled tree, where each label corresponds to a field name... Each scalar expression in the `SELECT` clause emits a value at the same level of nesting as the most-repeated input field used in that expression."
  - "So, the string concatenation expression emits `Str` values at the level of `Name.Language.Code` in the input schema. The `COUNT` expression illustrates within-record aggregation. The aggregation is done `WITHIN` each `Name` subrecord, and emits the number of occurrences of `Name.Language.Code` for each `Name`".
- Dremel's SQL dialect was a variant of SQL for semi-structured data, specifically protocol buffers, aka protobufs
  - Protobufs: typed, binary, schema-enforced alternative to JSON or XML.
  - Used pervasively at Google; nearly all data passed between applications or stored on disk used them.
  - Dremel's main contribution in this regard:
- If you remember our second lecture: semi-structured, hierarchical data has its downsides
  - Remember "what goes around comes around"?
  - Q: what were the downsides?
    - No normalization: redundancy across records can be large - update/deletion anomalies
    - Operating on nested data is difficult
  - But, can avoid joins
  - In fact, Dremel itself did not have joins for a while - limited joins after
- In contrast to OLAP, here's the OLTP story within Google:
  - F1 - SQL-based OLTP engine, started in 2009
    - Storage was on BigTable (remember this was an LSM-based KV store)
    - Use-case: ad backend - the big money maker
      - ads moved entirely to F1 in 2013
  - Spanner:
    - Generalized F1's capability for external audience
    - Offered on Cloud as Google Cloud Spanner
    - Notable for use of TrueTime (GPS+Atomic Clocks) to achieve linearizability - TODO: detour to cover linearizability?
  - AlloyDB:
    - More recent (from 2022) PostgreSQL-compatible HTAP service
- Several other SQL-based systems

    - PowerDrill, Procella, Tenzing, ...
    - More successful:
        - Cloud DataFlow - stream/batch processing service, generalizes other efforts like Kafka
- Standardization efforts run internally to ensure SQL dialects within Google are compatible
- All other popular noSQL OLAP systems eventually moved to SQL, including Hive, Spark, Presto, ...

Disaggregation

- Storage
    - Dremel began in 2006 as a shared-nothing parallel database (Teradata/Gamma origins)
    - Eventually Dremel evolved to work on disaggregated storage, specifically GFS
        - Each table (or portion of, aka a tablet) replicated k-way
        - Issues: "*... scanning a table consisting of hundreds of thousands of tablets required opening as many files in GFS, taking multiple seconds. Furthermore, the metadata format used by Dremel originally was designed for disk seeks as opposed to network roundtrips*."
            - In particular, opening a tablet file needed to do a RPC to the master for metadata + an RPC to the chunk server, and doing this hundreds of thousands of times in parallel created a metadata bottleneck (the single GFS master became a hotspot).
        - Solution:
            - changing storage format to be instead fewer, larger files to reduce metadata operations,
            - metadata representation (pre-fetching schemas and partition metadata in batch at the root server and pushing it down the execution tree),
            - query affinity (scheduling workers that are physically close to the data they read),
            - and prefetching (reading ahead to hide network latency).
- Memory
    - Original paper did not have joins
    - Later, added joins via shuffle
        - Originally used local RAM and disk "*to store sorted intermediate results. However, the tight coupling between the compute nodes and the intermediate shuffle storage proved to be a scalability bottleneck*"
        - Avoid having to store O(n) local memory and storage; instead transferring results across the network to distributed memory

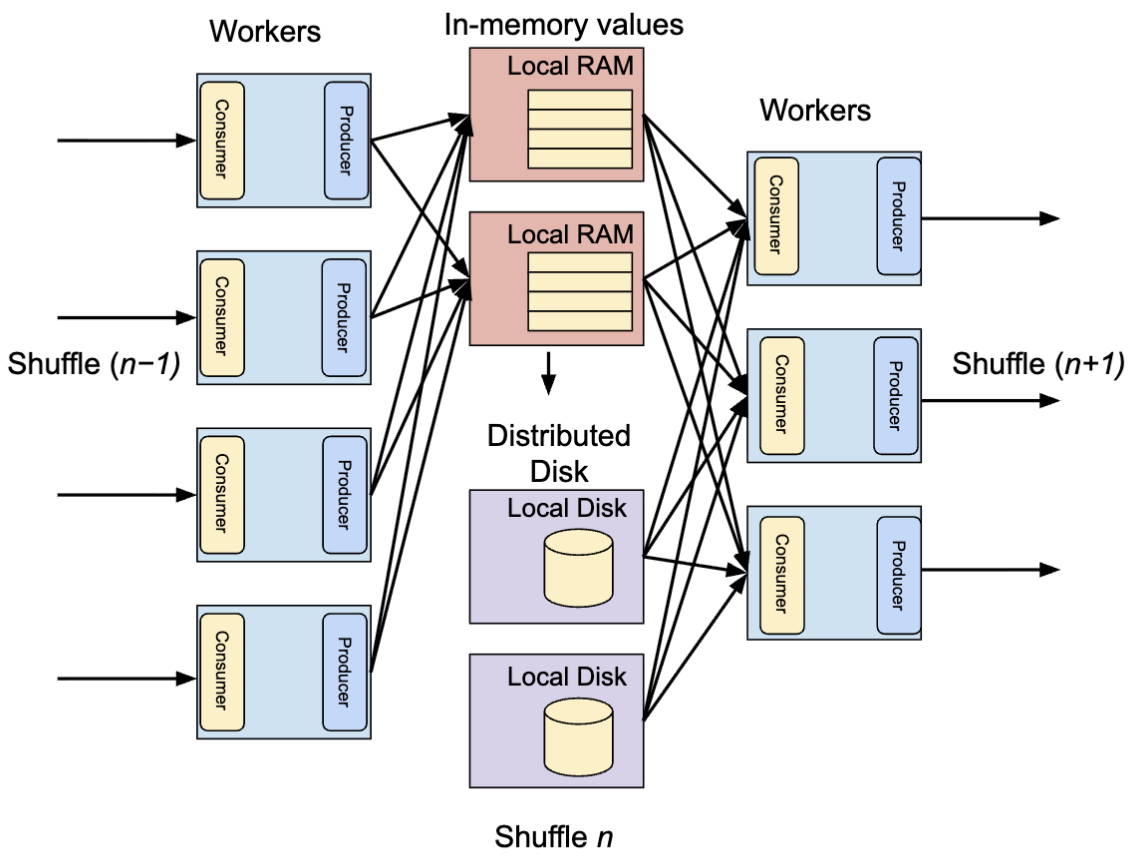

Figure 2: Disaggregated in-memory shuffle

    -
    - https://cloud.google.com/blog/products/bigquery/in-memory-query-execution-in-google-bigquery
    - "RAM and disk resources needed to store intermediate shuffle data were managed separately in a distributed transient storage system"
- Disaggregation as a broader trend:
  - Storage disaggregation:
    - used in transactional systems like Spanner, AWS Aurora
    - and analytical systems like Snowflake, Redshift
  - Storage can be accessed via higher level APIs and can be decoupled completely

Data Lake ("in situ") analysis

- Essentially, keep data in files, and query it (like other applications can), rather than in monolithic proprietary databases
- Jim Gray saw this coming in 2005 -> then MR essentially went further
- Various aspects to a data lake:
  - consuming data from a variety of data sources,
  - eliminating traditional ETL-based data ingestion from an OLTP system to a data warehouse, and
  - enabling a variety of compute engines to operate on the data.
- Instead, original shared-nothing design did not have this:
  - "*explicit data loading was required, and the data was stored in a proprietary format, inaccessible to other tools*"
- Built on top of protobufs
  - Protobufs had sharded files, and a schema definition

- Storage format:
  - Self-describing columnar format to support semi-structured data analysis
  - Another benefit: interoperability with other engines, e.g., MapReduce but also F1, Tenzing, PowerDrill, Procella
  - Since then, included support for other formats beyond the columnar protobuf format:
    - included Avro, CSV, JSON - row-based formats, no need to "ingest" (basically ingest routines as part of QP)
    - also included files on GCS/Google Drive
- Downside of "in-situ analysis"
  - Limited statistics (many queries on data "seen for the first time")
  - Limited optimization of storage layout
  - Limited access control/security
- Over time: *BigQuery Managed Storage*
  - Blends in-situ and managed storage
  - Similar ideas in Delta Lake, Apache Iceberg, Apache Hudi
    - These are "open table formats" that add:
      - transactional semantics,
      - schema evolution, and
      - storage layout optimization on top of files in object storage — giving the best of both worlds.

Serverless

- "*one of the pioneers in providing an elastic, multi-tenant, and on-demand service, now widely referred to in the industry as serverless*"
  - supporting interactive queries *plus* in situ analytics *plus* scaling to thousands of internal users at Google at a low cost ...
  - ... only possible if the service was multi-tenant and provided on-demand resource provisioning.
- To support this needed:
  - **Disaggregation**:
    - The disaggregation of compute, storage, and memory allows on-demand scaling and sharing of compute independently from storage.
    - Dremel started by decoupling disk storage from compute resources in 2009 and eventually added disaggregated memory in 2014.
  - **Fault tolerance and restartability**:
    - Each subtask is deterministic and repeatable;
    - The dispatcher can issue duplicate tasks for stragglers and reschedule failed tasks w/o full-query restart.
    - Makes it possible to share compute across users — a task can be preempted and restarted on a different machine.
  - **Virtual scheduling units (slots)**:
    - instead of allocating specific machine types, Dremel works with abstract compute+memory units called slots.
    - Decouples the customer-facing resource model from the physical cluster deployment. Slots remain the core customer-visible concept in BigQuery today.
- Query Execution (from original paper):
  - `SELECT A, COUNT(B) FROM T GROUP BY A` rewritten into
  - `SELECT A, SUM(c) FROM (R11 UNION ALL ... R1n) GROUP BY A`
    - where tables `R11, . . . , R1n` are the results of queries sent to the nodes `1, . . . , n` at level `1` of the serving tree:
      - `R1i = SELECT A, COUNT(B) AS c FROM T1i GROUP BY A`
- Evolution:
  - Using shuffle results to checkpoint
  - Flexible DAG scheduling, rather than the fixed tree:

    - COUNT - two levels sufficient
    - ORDER BY ... LIMIT (top-k) needs a deeper tree to avoid a bottleneck at the final merge
  - Dynamic execution: can change the query execution plan at runtime based on statistics collected during execution.
    - e.g., start with a hash join (shuffle both sides), but if one side finishes small, cancel the second shuffle and switch to a broadcast join instead.
    - avoids the notorious problem of cardinality estimation errors propagating exponentially through join trees.

Columnar Storage for Nested Data

- Context:
  - In the early 2000s, semistructured data (XML, JSON, protobufs) exploded. Google used protobufs extensively internally.
  - While columnar storage for flat relational data was well-known (C-Store / Vertica, MonetDB, etc.), ...
  - the Dremel paper pioneered columnar storage for semi-structured data.
- Encoding format:
  - Per column, encode all structure information *within* the column itself
  - Using two additional integer values per cell:
    - the **repetition level** (which repeated field in the path restarted) and
    - the **definition level** (how deep into the path are fields present, for optional fields that may be absent).
- Allows reading any individual leaf column without reading its ancestors — the column is self-contained.
- **Parquet** adopted this encoding - from Cloudera and Twitter, open-source
  - now the dominant columnar file format in the data lake ecosystem (used by Spark, Hive, Presto, DuckDB, etc.).
- Per group of rows (aka row group), store using this way...
- Example:

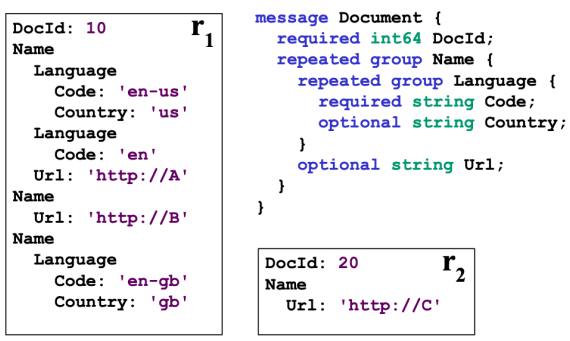

Figure 5: Two sample nested records and their schema (based on Figure 2 in [32])

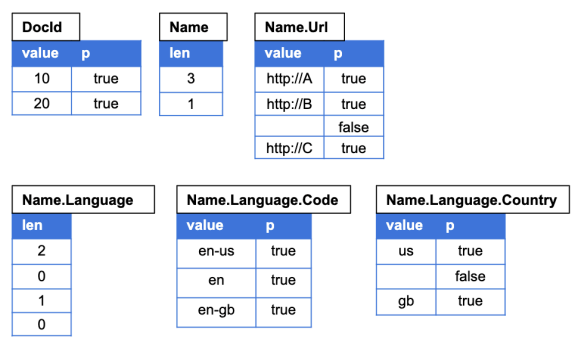

Figure 7: Columnar representation of the data in Figure 5 showing length (len) and presence (p)

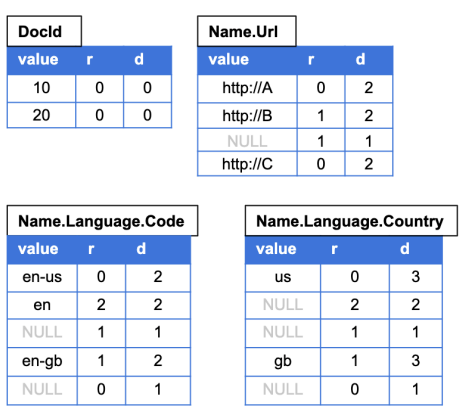

Figure 6: Columnar representation of the data in Figure 5 showing repetition levels (r) and definition levels (d)

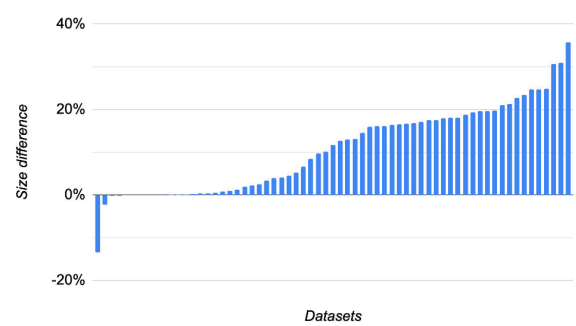

Figure 8: Percentage difference between repetition/definition and length/presence encodings. Higher bars correspond to smaller files with length/presence encoding.

-
  - Repetition: Which repeated ancestor started a new list at this value?

    - Definition: How far down the schema path is defined?
        - aka how many fields in path that could be undefined (because they are optional or repeated) are actually present?
    - For example, for `Name.Language.Country`
        - `us` - starting a new record so `0`. All of `Name` (repeated), `Language` (repeated), `Country` (optional) are defined, so `3`
        - `NULL` - this is for the potential `Country` after `en`. `Language` was repeated, so `2`. `Name` and `Language` are defined so `2`.
            - Since `d` < 3, value is NULL
        - `NULL` - this is for the potential `Country` under `http://B`. `Name` was repeated, so `1`. Only `Name` is defined, so `1`
        - `gb` - Name was repeated, so 1. `d = 3` because all are present
- Alternate representations:
    - **ORC**: tracks the *number of occurrences* of repeated fields within each parent (lengths), and boolean presence flags for optional fields.
    - **Arrow**: similar to ORC but tracks repeated fields by their cumulative *offsets* rather than lengths, facilitating direct array element access (makes sense for an in-memory format).
- Tradeoff with length/presence:
    - To read a leaf column, you must also read all its ancestor columns (to reconstruct boundaries).
    - But the encoding is more compact: the repetition/definition scheme redundantly re-encodes ancestor structure for every child, so deeper and wider schemas have more redundancy.
    - Analysis across 65 internal datasets:
        - length/presence encoding was *on average 13% smaller* than repetition/definition level encoding.
    - Tradeoff:
        - repetition/definition levels allow single-column access at the cost of storage overhead;
        - length/presence requires ancestor reads but compresses better.
- Improvements with Capacitor (2014+)
    - Columnar techniques
        - Predicate pushdown into the representation, with bounds per row group (min/max value)
        - Vectorized evaluation
        - Skip-indexes (segmented column layout with an offset index to skip non-matching segments without decompressing them), and
        - Predicate reordering heuristics.
        - *Row reordering*:
            - Permutes rows to improve run-length encoding (RLE) effectiveness.
            - Finding the optimal permutation is NP-complete, so Capacitor uses sampling and heuristics.
            - In practice 17% average size reduction across 40 internal datasets, with some reaching 40–75%.

Interactive Query Latency over Big Data

- The design principles (disaggregation, in situ processing, serverless) are individually counterproductive to low latency:
    - disaggregation adds network round trips;
    - in situ processing makes storage layout optimization hard;
    - serverless shared resources are less predictable than dedicated machines.
- Dremel addressed this with various latency-reduction techniques:
    - Columnar representation avoided processing unnecessary columns; compression helped reduce data read; DAG/tree helped parallel processing; in addition --
    - *Stand-by server pool*:

    - a distributed SQL engine kept ready to process queries immediately — eliminating the per-job startup latencies of MapReduce jobs.
  - *Speculative execution*:
    - break the query into thousands of small tasks; workers pick up tasks as they complete, so slow machines do fewer tasks and fast machines do more.
    - For long-tail stragglers, issue duplicate tasks; performance becomes a function of total available resources rather than the slowest component.
  - *Approximate results*: Dremel allowed use of approximation (e.g., hyperloglog) and also setting of fraction of data after which answers can be returned (e.g., 98%)
  - *Query latency tiers*: preempts parts of large queries to allow small/interactive queries to proceed without monopolization.
  - *Reuse of file operations*: batch-fetch file metadata at the root server and push it down the execution tree to avoid thousands of concurrent RPC calls to the GFS master; use larger files to reduce total metadata operations
- From original paper:

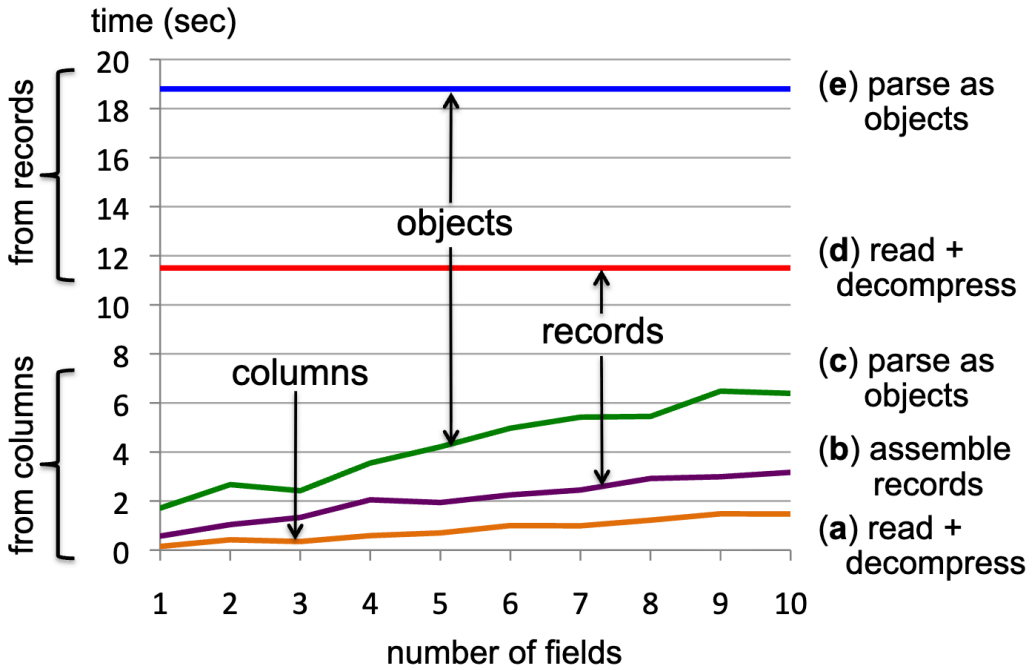

Figure 9: Performance breakdown when reading from a local disk (300K-record fragment of Table T_1)

Snowflake paper

- Implemented on top of commercial cloud (Amazon); began 2012, GA in 2015
- SaaS, aka Users need not buy machines, hire database administrators, or install software.
- Also, like Dremel
  - Semi-Structured + Columnar
  - "Elastic Storage and compute resources can be scaled independently and seamlessly, without impact on data availability or performance of concurrent queries."
  - Available and durable
- Disaggregated storage: built on Amazon's S3
  - Compute is provided through Snowflake's (proprietary) shared-nothing engine.

    - To reduce network traffic between compute nodes and storage nodes, each compute node caches some table data on local disk. An added benefit of this solution is that local disk space is not spent on replicating the whole base data, which may be very large and mostly cold (rarely accessed). Instead, local disk is used exclusively for temporary data and caches, both of which are hot (suggesting the use of high-performance storage devices such as SSDs). So, once the caches arewarm, performance approaches or even exceeds that of a pure shared-nothing system. We call this novel architecture the multi-cluster, shared-data architecture.
    - S3: HTTP(S)-based PUT/GET (range) /DELETE interface. Objects i.e. files can only be (over-)written in full.
    - Therefore: Tables are horizontally partitioned into large, immutable files which are equivalent to blocks or pages in a traditional database system
        - Within each file, the values of each attribute or column are grouped together and heavily compressed via PAX
        - Each table file has a header which, among other metadata, contains the offsets of each column within the file. Because S3 allows GET requests over parts of files, queries only need to download the file headers and those columns they are interested in.
- Disaggregated compute: built on Amazon's EC2
    - Virtual Warehouses: VWs are pure compute resources. They can be created, destroyed, or resized at any point, on demand. Creating or destroying a VW has no effect on the state of the database.
    - Various sizes
- Execution:
    - Columnar, Vectorized,
    - Push-based
        - ":push-based execution improves cache efficiency, because it removes control flow logic from tight loops"
        - "t also enables Snowflake to efficiently process DAG-shaped plans, as opposed to just trees, creating additional opportunities for sharing and pipelining of intermediate results."
    - Cascades-style QO
- Transactions: MVCC + SI
    - MVCC natural because files are immutable on S3
- No indexes, but maintain statistics per page:
    - Min-max aka small materialized aggregates/zone-maps
    - "the system maintains the data distribution information for a given chunk of data (set of records, file, block etc.), in particular minimum and maximum values within the chunk."
- Semi-structured support
    - Pulls common columns into their own columnar format instead of storing them as parquet
- Contrast to BQ:
    - "One of the inspirations for Snowflake was BigQuery’s support for JSON and nested data, which we find necessary for a modern analytics platform. But while BigQuery offers a SQL-like language, it has some fundamental deviations from the ANSI SQL syntax and semantics, making it tricky to use with SQL-based products. Also, BigQuery tables are append-only and require schemas. In comparison, Snowflake offers full DML (insert, update, delete, merge), ACID transactions, and does not require schema definitions for semi-structured data."

Finally, some thoughts about complexity

- From https://www.vldb.org/pvldb/vol18/p5126-bress.pdf

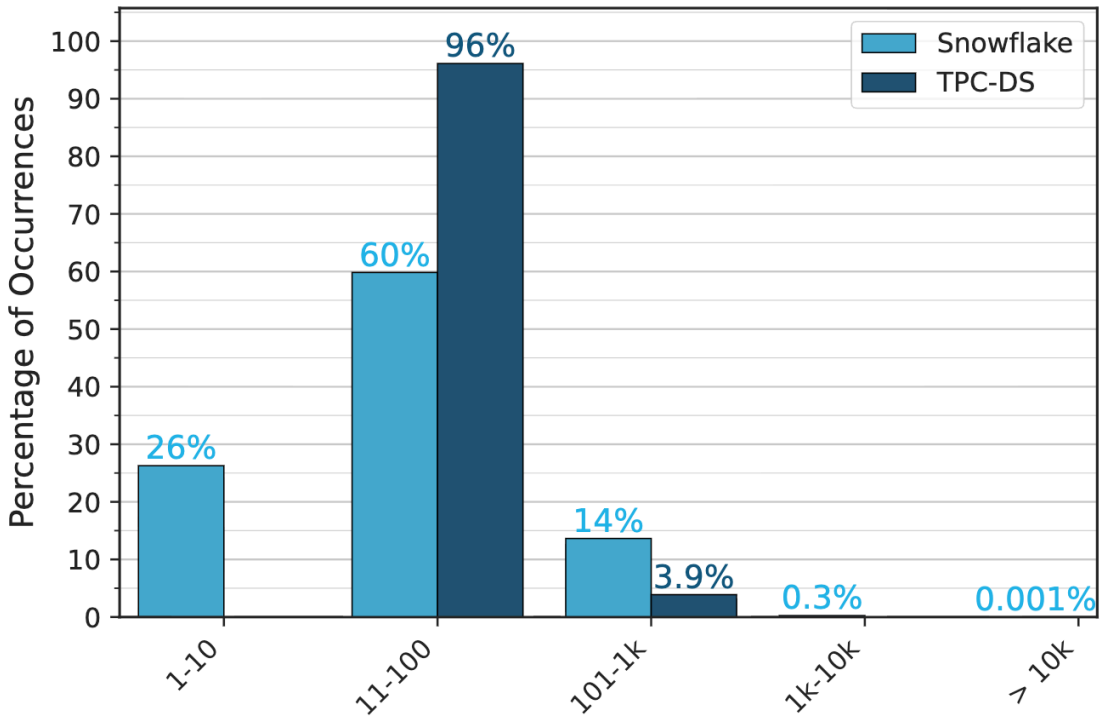

Figure 3: Distribution of Number of Operators per Query

Table 4: Distribution of Projection and Filter Expressions

Projection Expressions			
Category	**1–10**	**11–100**	**101–1K**
Snowflake	40%	46%	14%
TPC-DS	54%	45%	1%
Filter Expressions			
Category	**1–10**	**11–100**	**101–1K**
Snowflake	24%	65%	11%
TPC-DS	54%	45%	1%

Table 8: Distribution of Number of Joins per Query for Snowflake (SF) and TPC-DS (DS) Workload

Range	SF (%)	DS (%)	Range	SF (%)	DS (%)
1-2	34.0	9.0	31-50	1.8	0.0
3-5	27.0	45.0	51-100	1.0	0.0
6-10	19.0	32.0	101-500	0.5	0.0
11-20	12.0	13.0	501-1000	0.02	0.0
21-30	5.0	1.0	1000+	0.0003	0.0

Table 13: Aggregation Group By Keys

	0	1-2	3-5	6-10	10+
Snowflake (%)	7	53	26	7	7
TPC-DS (%)	8	56	28	7	0

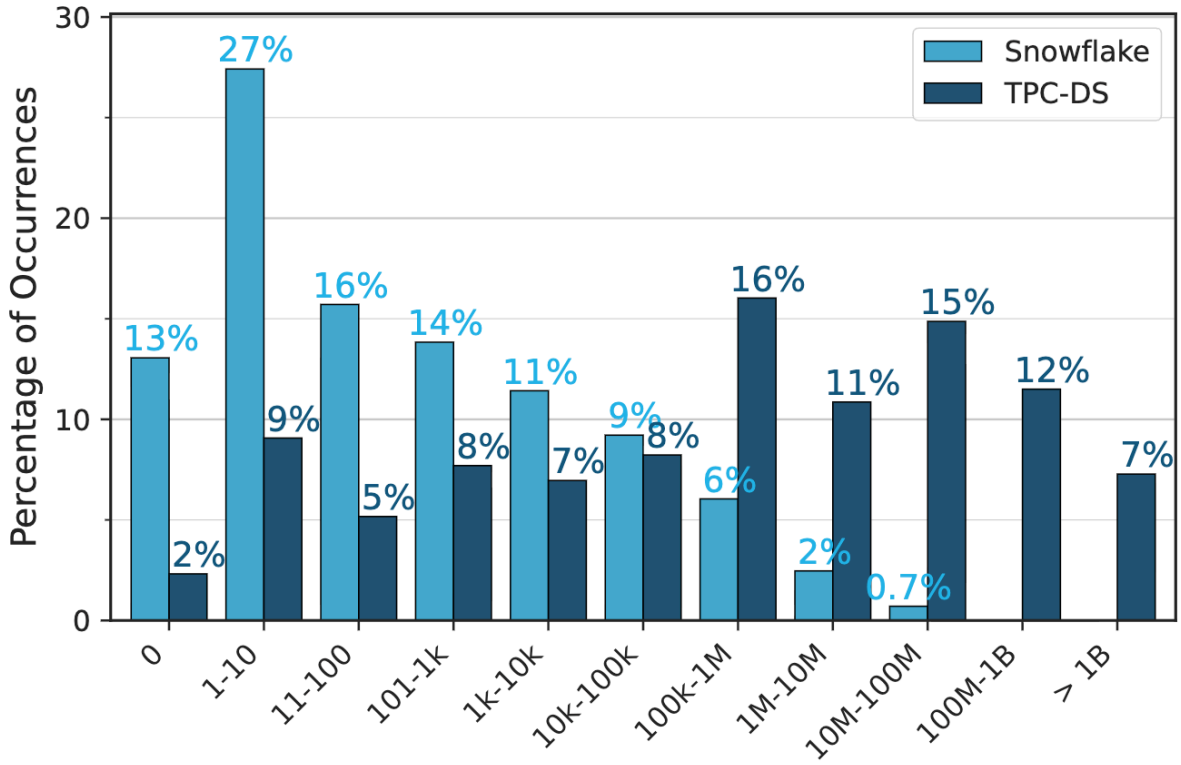

Figure 10: Number of Distinct GroupBy Keys